Smart Traffic Signal Optimization

Machine learning and computer vision | Detailed project explanation

Abstract

FlowState is an educational prototype that investigates adaptive control of a four-way traffic intersection. A trained Random Forest predicts short-term vehicle arrivals. A rule-based controller combines these forecasts with queue length and time since green to choose a signal phase. A separate computer vision demonstration detects and counts vehicles in generated images.

The project compares fixed-time, queue-only adaptive and ML-assisted control using identical synthetic traffic. It provides a local Python backend, an interactive browser dashboard, reproducible evaluation files and presentation slides. Results show that adaptation provides most of the measured benefit; forecasting contributes a smaller average improvement.

Problem and objective

An equal fixed timer cannot react to uneven demand. A lightly used approach can receive green while a busier approach builds a queue. The objective is to measure whether demand-responsive control reduces accumulated waiting and increases completed crossings in a controlled simulation.

System architecture

Component	Input	Output
Traffic generator	Scenario, random seed	Arrivals for each approach
Random Forest	Three recent arrival counts	Next 10-second arrival forecast
Signal controller	Queue, forecast, waiting age	Next green approach
Computer vision demo	Generated RGB frame	Boxes and visible counts
Dashboard	Simulation and model outputs	Animation, charts and metrics

Two separate paths:
Control: simulated arrivals → forecast + queue state → signal decisions.
Vision: synthetic image → segmentation → bounding boxes and counts.

What is learned, and what is designed?

The arrival forecaster is learned from data. Signal selection is a manually designed heuristic. Vision uses classical pixel processing. The project does not train a signal policy, use reinforcement learning, or include a pretrained real-road detector. The vision output is not currently connected to the controller.

Dataset & feature design

Synthetic traffic generation, example construction and train/test separation

Generating traffic episodes

Vehicle counts are sampled from a Poisson distribution: **N ~ Poisson(10 x rate)**, where rate is the expected arrivals per second and N is the arrival count during ten seconds. This produces naturally variable counts even when the underlying average demand is steady.

Each episode begins with a random rate between 0.04 and 0.65 vehicles/second. Every six intervals, including the first interval, a normally distributed change with standard deviation 0.09 is applied; the rate is clipped to 0.03-0.70. Each episode contains 36 ten-second intervals, representing six minutes.

Input features and target

Feature	Time interval relative to prediction
arrivals_last_10s	Most recent completed 10 seconds
arrivals_previous_10s	10-20 seconds ago
arrivals_previous_20_to_30s	20-30 seconds ago
Target	Arrivals in the next 10 seconds

If three consecutive observed intervals contain 2, 4 and 5 vehicles in chronological order, the input is **[5, 4, 2]**, because the newest count comes first. The following observed interval supplies the training label. This is a numeric prediction problem, so it is regression rather than classification.

Dataset sizes

Split	Episodes	Examples	Generator seed
Training	300	9,900	21
Held-out testing	100	3,300	722

An episode of 36 intervals produces 33 input-target pairs after reserving the first three intervals as history. Thus 300 x 33 = 9,900 training examples. Overlapping windows within an episode are allowed, but an entire episode belongs to only one split.

Preventing future information leakage

Training and test episodes are independent. In simulation, the prediction at time t reads only arrivals before t, never the next interval. An automated test records the exact model inputs and checks this boundary. At startup, missing history is zero-filled, creating a brief cold-start condition.

Scope of the dataset

The generator is useful for a repeatable coursework experiment, but it does not reproduce all real traffic patterns. It omits correlated platoons, road geometry, incidents, weather and driver behavior. Test performance on this generator does not establish accuracy on real intersections.

Model & training algorithm

Random Forest Regression implemented in scikit-learn

How a decision tree learns

A regression tree learns threshold questions about the input features. At each split, the algorithm searches for a feature and threshold that reduce squared prediction error. Samples are divided into two groups, and the process repeats until a stopping condition is reached. A leaf predicts the average target of the training examples that reach it.

How the forest learns

Each tree is trained using a bootstrap sample: a random sample of training rows drawn with replacement. Different samples produce different trees. Predictions from all trees are averaged, reducing sensitivity to the behavior of any single tree. This project uses scikit-learn defaults for unspecified settings, including squared-error splitting and bootstrap sampling.

Hyperparameter	Value	Purpose
n_estimators	60	Number of trees averaged
max_depth	8	Limits tree complexity
min_samples_leaf	15	Avoids overly specific leaves
random_state	9	Reproducible forest construction
n_jobs	1	One worker

Training procedure

1. Generate training episodes and build feature-target pairs.
2. Construct RandomForestRegressor with the configuration above.
3. Call model.fit(X_train, y_train).
4. Predict on independent held-out episodes.
5. Calculate MAE and RMSE and compare with simple forecasts.

No normalization is applied. Tree thresholds work directly on arrival counts. There are no neural-network epochs, learning rate, backpropagation or gradient-descent training in this model. The Python application trains the forest on the first model request; predictions are then reused during experiments.

Interpreting the output

Recent counts (newest first)	Predicted next 10-second arrivals
[0, 0, 0]	0.544
[1, 1, 1]	1.237
[3, 3, 3]	3.449
[6, 6, 6]	5.608
[10, 10, 10]	6.882

Outputs are expected counts and may be fractional. A positive prediction after three empty intervals represents possible future arrivals. Forests do not reliably extrapolate beyond their training distribution; unusually high counts can lead to conservative forecasts.

Adaptive signal algorithm

Forecasts inform a heuristic; the model does not directly select signal colors

Priority score

Priority = Q + 0.7 x P + 0.08 x T
Q = current queue length
P = predicted arrivals over the next 10 seconds
T = seconds since that approach last had green

A large queue increases urgency, the forecast anticipates near-term demand, and waiting age discourages repeated neglect of a quiet approach. The weights 0.7 and 0.08 are manually chosen design values. They are not fitted by the Random Forest or established as optimal.

Worked example

For a queue of 12 vehicles, forecast of 5 vehicles and waiting age of 30 seconds, the score is 12 + 0.7 x 5 + 0.08 x 30 = **17.9**. A competing approach with queue 4, forecast 2 and age 50 scores **9.4**. The first approach has higher normal priority, subject to timing and overdue rules.

Decision procedure

Every simulated second: add arrivals and update time since green. Every ten seconds, refresh forecasts from past arrivals. If a clearance transition is active, discharge no vehicles. Otherwise discharge up to one vehicle from the active approach and evaluate whether its green should end.

A waiting approach with at least 90 seconds since green receives overdue priority. Otherwise, choose the highest-scoring approach. A normal early change requires a different approach whose score exceeds the active score by more than 2. Green must first last at least 10 seconds. At 45 seconds, switch to another approach even if the current approach still has the highest score.

Parameter	Value / behavior
Minimum / maximum green	10 / 45 seconds
Transition	3 seconds without vehicle discharge
Overdue handling	Priority after 90 seconds without green, at an eligible decision
Discharge rate	Up to one vehicle/second from one approach
Fixed-time baseline	N, E, S, W; 25-second green each, with transitions

Why include queue-only adaptation?

The queue-only policy keeps the adaptive rules but sets forecast contributions to zero. Comparing it with ML-assisted control isolates the incremental role of forecasting. Comparing only against fixed timing could incorrectly credit the ML model for improvements mainly caused by adaptation.

Overdue handling reduces starvation but is not a formal 90-second bound on vehicle waiting. These rules are educational assumptions, not a certified controller for physical traffic lights.

Computer vision & interface

A transparent demonstration of detecting vehicles from image pixels

Detection pipeline

The camera generator draws separated blue vehicle rectangles on a 320 x 320 RGB road image. The detector receives this pixel array, not ground-truth bounding boxes. It performs the following steps:

Stage	Implementation
Color mask	Select pixels where blue > 160, green > 120 and red < 130.
Connected components	Flood fill using four-neighbor adjacency: up, down, left and right.
Noise filtering	Reject components containing fewer than 50 pixels.
Bounding box	Find minimum and maximum x/y coordinates of each component.
Approach assignment	Use the component center and fixed road regions to assign N/E/S/W.
Output	Return boxes and approach counts for display.

What this demonstrates

The module demonstrates image segmentation, spatial grouping, bounding-box extraction and counting. It uses NumPy arrays and a custom flood-fill implementation, rather than OpenCV or a trained neural detector. Tests verify pixel-derived counts for empty, mixed and capacity-limited synthetic scenes.

Important integration boundary

Camera counts do not drive the current signal controller. The simulator knows complete queues and arrivals. The camera panel separately demonstrates counting and shows at most nine vehicles per approach. It cannot represent the full length of a long queue.

Reading the dashboard

Digital twin: an animated four-way intersection with queues and signal state. Playback can be paused or scrubbed.
Camera panel: image-based detections with boxes and visible approach counts.
Comparison chart: total queued vehicles over time for fixed and ML-assisted policies.
Decision engine: predicted arrivals for each approach.

Headline metrics summarize the full 600-second run, even when playback is at an earlier point. The methodology view displays all three controller results. The application also contains nine presentation slides and speaker notes.

What real-video integration would require

Real roads need a validated detector, tracking to avoid double counting, calibrated approach regions, queue and arrival estimation, and handling for occlusion and changing lighting. Those capabilities are future work and are not claimed by this prototype.

Experimental results

Forecast quality and control quality are separate measurements

Forecast performance on 3,300 held-out examples

Predictor	MAE (vehicles / 10 seconds)
Random Forest	1.704
Mean of three recent counts	1.733
Latest-count persistence	2.061
Constant training-set mean	2.198

MAE is the average absolute difference between predicted and observed counts. **RMSE** is the square root of average squared error; the forest scores 2.203 vehicles. The forest beats these baselines on this synthetic test set, but its gain over a moving average is modest.

Controller evaluation protocol

Each run lasts 600 seconds. Three scenarios are evaluated using seeds 0-9, producing 30 paired experiments and 90 policy runs. Every policy receives the exact same arrival array for a scenario and seed. Results below are means and standard deviations across ten seeds, not confidence intervals.

Scenario	Fixed delay	Queue-only delay	ML-assisted delay
Balanced	38.78 +/- 2.23 s	21.17 +/- 1.60 s	21.11 +/- 1.81 s
N/S rush	99.39 +/- 12.69 s	51.29 +/- 10.46 s	49.10 +/- 10.84 s
East surge	93.34 +/- 8.92 s	33.39 +/- 4.51 s	32.17 +/- 3.98 s

Delay per arrival = accumulated queue-seconds / arriving vehicles. This includes waiting accumulated by unfinished vehicles during the run, but not their future waiting after the run ends. A departed-only average could hide long queues left behind.

Mean vehicles served	Fixed-time	Queue-only	ML-assisted
Balanced	351.7	362.0	362.7
N/S rush	337.0	413.8	416.2
East surge	279.9	356.2	356.7

What the findings support

Adaptation supplies most of the observed improvement. Forecasting adds a smaller average gain, especially with asymmetric demand. The balanced-scenario difference is very small. No statistical significance, universal superiority or real-world percentage reduction is claimed.

An honest counterexample: for rush traffic with seed 42, delay is 54.97 s with ML and 54.40 s with queue-only control. ML changes decisions but is slightly worse on this run. Its multi-seed mean is better. This is why repeated evaluation matters.

Implementation & verification

Running, reproducing and understanding the source code

Technology and files

File / technology	Responsibility
Python + NumPy + scikit-learn	Traffic generation, numeric processing and model training
app.py	Local HTTP server and API; trains on the first model request
src/model.py	Episode generation, forest training and forecast metrics
src/simulation.py	Queues, signal policies and shared-arrival comparison
src/vision.py	Synthetic images, pixel segmentation and detections
web/	HTML/CSS/JavaScript dashboard and presentation
evaluate.py	Multi-seed controller results in CSV and JSON
verify_model.py	Model sanity checks and controller integration evidence

Run the application

From the project directory, run **python3 app.py**, then open **http://localhost:8000**. Keep the terminal running during your presentation. Once dependencies are installed, the local app works without internet. On a new computer, install requirements.txt inside a Python virtual environment first.

Reproduce the evidence

```
python3 -m unittest discover -s tests -v
```

```
python3 evaluate.py
```

```
python3 verify_model.py
```

The artifacts folder contains model_metrics.json, model_verification.json, evaluation.csv, evaluation_summary.json and environment.json. These record model errors, prediction examples, controller outcomes and the library versions used.

Verification checks

Tests verify: reproducible arrivals; conservation of arrivals = served + remaining; zero queues for empty traffic; synthetic image counts; no additional departures between sampled states entirely inside transitions; forecast improvement over persistence; finite predictions that respond to demand; past-only model inputs; and changed signal decisions when learned forecasts are included.

Additional verification confirms 60 fitted trees and evaluates stronger forecasting baselines. In the rush seed-42 run, enabling the learned forecast changes 199 sampled signal states relative to queue-only control. This demonstrates actual integration, not proof that every changed decision is better.

Verification limits

Python tests and browser interaction checks passed. The dashboard was visually inspected at both narrow and desktop widths. Tests validate selected properties, not physical-road safety, universal accuracy or all possible software edge cases.

Presentation & next steps

Explain the project clearly without overstating what it proves

A short explanation you can say aloud

"My project uses Random Forest Regression to predict vehicle arrivals in the next ten seconds. A heuristic controller combines the forecast with current queues and time since green to choose signal phases. I compare fixed-time, queue-only and ML-assisted policies using identical synthetic traffic, and separately demonstrate vehicle counting through image processing."

Common viva questions

Why Random Forest? It is practical for small tabular datasets, captures nonlinear relationships and requires little feature preprocessing. This project does not prove it is the best possible model.

Where is machine learning? In the arrival forecaster. Its trees learn from input-target examples. The priority formula and timing rules are manually designed.

Is this reinforcement learning? No. There is no learned action policy, reward function or policy training. This is supervised forecasting plus heuristic control.

Why use a queue-only baseline? It separates the benefit of adapting to queues from the additional benefit of predicting arrivals.

What is data leakage? Using information during training or prediction that would not legitimately be available. Separate episodes avoid train/test overlap, and simulation forecasts use only past arrivals.

Does the camera control the lights? Not in this version. Camera processing is a separate synthetic demonstration. Full simulation counts drive the controller.

Can ML perform worse? Yes. Counts are noisy and a useful forecast does not guarantee a better control decision. Report repeated trials, not only a favorable run.

Limitations and development roadmap

Data: replace synthetic assumptions with measured demand and evaluate distribution shift.
Vision: add real-video detection, tracking and calibrated queue estimates.
Simulation: model travel time, turning movements, pedestrian phases and realistic discharge rates.
Evaluation: test more seeds, tune on separate validation scenarios, analyze weights and report paired uncertainty.
Deployment: add failure handling and traffic-engineering safety validation before any physical use.

Source of this report

This report describes the implemented project, not a literature survey. Numerical evidence comes from artifacts/model_verification.json and artifacts/evaluation_summary.json. Algorithm details follow src/model.py, src/simulation.py and src/vision.py. Reproduction commands are on page 7. The current deliverable is a local academic prototype; this report does not assert that online deployment is complete.

Main conclusion: the model is trained and working, its forecasts affect signal decisions, and the simulator supports a fair comparison. The strongest observed benefit is adaptive control, with a smaller contribution from ML forecasting.